

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def encrypt(text, shift):  
    result = ""  
    for ch in text.upper():  
        if ch.isalpha():  
            result += chr((ord(ch) - 65 + shift) % 26 + 65)  
        else:  
            result += ch  
    return result
```

```
def decrypt(text, shift):  
    result = ""  
    for ch in text:  
        if ch.isalpha():  
            result += chr((ord(ch) - 65 - shift) % 26 + 65)  
        else:  
            result += ch  
    return result  
  
plain = input("Enter Plaintext : ")  
shift = int(input("Enter Key : "))  
cipher = encrypt(plain, shift)  
print("Ciphertext :", cipher)  
original = decrypt(cipher, shift)  
print("Decrypted :", original)
```

OUTPUT:

```
Enter Plaintext : HELLO  
Enter Key : 3  
Ciphertext : KHOOR  
Decrypted : HELLO
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
from string import ascii_uppercase

def generate_key_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()
    for ch in key + ascii_uppercase:
        if ch == 'J':
            continue
        if ch not in used:
            used.add(ch)
            matrix.append(ch)
```

```
return [matrix[i:i+5] for i in range(0, 25, 5)]
```

231401102

```

def find_position(matrix, ch):
    if ch == 'J':
        ch = 'I'
    for i, row in enumerate(matrix):
        if ch in row:
            return i, row.index(ch)

def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())
    result = ""
    i = 0
    while i < len(text):
        a = text[i]
        if i + 1 < len(text):
            b = text[i + 1]
            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1
    return result

def encrypt(text, matrix):
    text = prepare_text(text)

```

```

cipher = ""
for i in range(0, len(text), 2):
    a = text[i]
    b = text[i + 1]
    r1, c1 = find_position(matrix, a)
    r2, c2 = find_position(matrix, b)
    if r1 == r2:
        cipher += matrix[r1][(c1 + 1) % 5]
        cipher += matrix[r2][(c2 + 1) % 5]
    elif c1 == c2:
        cipher += matrix[(r1 + 1) % 5][c1]
        cipher += matrix[(r2 + 1) % 5][c2]
    else:
        cipher += matrix[r1][c2]
        cipher += matrix[r2][c1]
return cipher

def decrypt(cipher, matrix):
    plain = ""
    for i in range(0, len(cipher), 2):
        a = cipher[i]
        b = cipher[i + 1]
        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)
        if r1 == r2:
            plain += matrix[r1][(c1 - 1) % 5]
            plain += matrix[r2][(c2 - 1) % 5]
        elif c1 == c2:

```

```

        plain += matrix[(r1 - 1) % 5][c1]
        plain += matrix[(r2 - 1) % 5][c2]
    else:
        plain += matrix[r1][c2]
        plain += matrix[r2][c1]
        cleaned = ""
        i = 0
    while i < len(plain):
        if (i + 2 < len(plain))
            and plain[i] == plain[i + 2]
            and plain[i + 1] == 'X'):
            cleaned += plain[i]
            i += 2
        else:
            cleaned += plain[i]
            i += 1
    if cleaned.endswith('X'):
        cleaned = cleaned[:-1]
    return cleaned

key = input("Enter Keyword : ")
plain = input("Enter Plaintext : ")
matrix = generate_key_matrix(key)
print("\nKey Matrix")

for row in matrix:
    print(" ".join(row))

cipher = encrypt(plain, matrix)
print("\nCiphertext :", cipher)

```

```
original = decrypt(cipher, matrix)
```

```
print("Decrypted Text :", original)
```

OUTPUT:

Enter Keyword : PLAY

Enter Plaintext : HELLO

Key Matrix

P L A Y B

C D E F G

H I K M N

O Q R S T

U V W X Z

Ciphertext : KCYVPQ

Decrypted Text : HELLO

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import numpy as np
key = np.array([[3, 3], [2, 5]])
inverse = np.array([[15, 17], [20, 9]])
def process(text):
    text = text.upper()
    if len(text) % 2 != 0:
```

```

        text += "X"

    return text

def convert(text):

    return [ord(i) - 65 for i in text]

def encrypt(text):

    text = process(text)

    nums = convert(text)

    cipher = ""

    for i in range(0, len(nums), 2):

        pair = np.array([[nums[i]], [nums[i + 1]]])

        result = np.dot(key, pair) % 26

        cipher += chr(result[0][0] + 65)

        cipher += chr(result[1][0] + 65)

    return cipher

def decrypt(cipher):

    nums = convert(cipher)

    plain = ""

    for i in range(0, len(nums), 2):

        pair = np.array([[nums[i]], [nums[i + 1]]])

        result = np.dot(inverse, pair) % 26

        plain += chr(result[0][0] + 65)

        plain += chr(result[1][0] + 65)

    return plain

plain = input("Enter Plaintext : ")

cipher = encrypt(plain)

print("Ciphertext :", cipher)

print("Decrypted :", decrypt(cipher))

```

OUTPUT:

Enter Plaintext : HELP
Ciphertext : HIAT
Decrypted : HELP

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def generateKey(text, key):  
    key = key.upper()  
    while len(key) < len(text):  
        key += key  
    return key[:len(text)]  
  
def encrypt(text, key):
```

```

cipher = ""
for i in range(len(text)):
    x = (ord(text[i]) - 65 + ord(key[i]) - 65) % 26
    cipher += chr(x + 65)
return cipher
def decrypt(cipher, key):
    plain = ""
    for i in range(len(cipher)):
        x = (ord(cipher[i]) - 65 - (ord(key[i]) - 65)) % 26
        plain += chr(x + 65)
    return plain
plain = input("Enter Plaintext : ").upper()
keyword = input("Enter Keyword : ")
key = generateKey(plain, keyword)
cipher = encrypt(plain, key)
print("Generated Key :", key)
print("Ciphertext :", cipher)
original = decrypt(cipher, key)
print("Decrypted :", original)

```

OUTPUT:

```

Enter Plaintext : ATTACKATDAWN
Enter Keyword : LEMON
Generated Key : LEMONLEMONLE
Ciphertext : LXFOPVEFRNHR
Decrypted : ATTACKATDAWN

```

RESULT: